UNDERGRAD CV

Newton Niu

Year 1 · BSc Computer Science · The Hong Kong Polytechnic University

github.com/Newton-666
Email: newt2022@qq.com
Phone: +86 13482829413

SUMMARY

I am a first-year Computer Science student at The Hong Kong Polytechnic University. I build personal and multi-agent systems with AI coding tools, but the work I care about is not generating code, it is deciding what the system is allowed to believe, when a failure is real, and how knowledge should move between context and long-term memory. I prefer to measure before I conclude, and to keep feature growth from outrunning stability.

EDUCATION

I study BSc Computer Science at The Hong Kong Polytechnic University (Year 1; expected graduation 2030). Alongside coursework I keep a living design journal in Obsidian and use several AI coding agents as a deliberate workflow.

PROJECTS

Bobo Agent , personal AI agent for knowledge work

github.com/Newton-666/boboagent · Python · LLM APIs · Obsidian

The philosophy of Bobo came from asking what would make an agent *mine* (my personal working buddy). The time I've built it is when I am reviewing for my final high school exam (IBDP). I wanted it to live inside Obsidian so that work, memory, and action stay one loop; each skill is being created as a markdown file so that I could edit easily with Obsidian. Bobo should do what it *learns*. That pushed me toward entropy thinking: if features arrive faster than I can verify them, the system collapses under its own options. Therefore, I start asking the question “why would programs developed by AI tools and vibe coding break if LLM is already capable enough to handle Python?” Later on, when I debug with other models, I refuse speculative bug lists. I often think debugging in this way: whenever there is a bug arriving that breaks the system, then there will be at least two or more smaller issues within other modules; I called that a *chain effect*. The solution that I determined later was to run all tests that I've created through vibe coding and run the smoke test to ensure that the program could indeed work.

Krystal (zebra-tui) , multi-agent terminal cockpit

github.com/Newton-666/zebra-tui · TypeScript · tmux · agent orchestration

The philosophy of Krystal came from watching products like Grok Bot and asking why agent-to-agent talk still needs a human as the router. I built on tmux so that communication and live panes could run automatically without copy and pasting. The harder question was the memory. Traditional memory systems was labeling information that the user quoted using “remember this” in a RAG system. The issue for this is that RAG are way too complicated and it suffers at extracting memory index at correct occasions; it only judge based on a similarity value. Once, when I see a report written by Robert Desimone from MIT McGovern Institute, the most critical thing that makes human memory interesting is the consciousness. Therefore I stopped treating “the agent forgot” as a storage shortage and started treating it as a signal to applied scene mapping. Each time when a memory is being indexed, it will add the scene where it was applied. Therefore the dimension of decision will ensure a more precise result than using RAG vectors itself. The enhanced tmux system presents my thinking in multi-agent orchestrations and communication. Whereas Krystal Bot is my new interpretation of agent as a neuroscience-driven specie after Bobo.

Sonic Lab , AI mix console for independent creators

Web Audio · local audio ML experiments · AI chat controlling a desk

Sonic Lab is the first machine learning experiment that I've done by myself. The inspiration for this project is when I am recording my vocals, there are environmental noise. Therefore I want to listen to a clearer vocal rather than purchasing a microphone. I want the ML model to contain generalizability when facing a new audio, therefore I cannot flatten every vocal into one “clean” answer. I treat artistic marks (rasp, breath, a loose groove) as identity (I called it sound features or latent variables), so the hard problem is knowing when *not* to fix. That leads to three product requirements: AI suggests rather than decides, every process stays 0-100% so the artist can keep the flaw, and human character must not be optimized away. However, if all features were suggested by AI, then it wouldn't become a ML model, but a chat. To resolve this problem, I've gathered 200 lossless mp3 audios and around 50 live audios. From a model perspective, the amount of data was way too less to ensure generalizability. However, the result from this training is that the audio could separate really well for trained audio, but relatively worse for unseen audios. In this project, I've learnt the essential component of machine learning, which is Data, especially the significance of it to ensure that a model have seen all possible situations to make accurate operations for new inputs.

SKILLS

- Creating web and model projects using vibe coding
- Visual design for the web
- User interface design
- Interactive design

INTERESTS FOR INTERNSHIP

I want to bring the same habit (clear interaction rules, measurable prototypes, and memory/context discipline) to **AI-supported art understanding and appreciation**.